

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Машковцева Марина

К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Разделить монолитное бэкенд-приложение (сервис обмена рецептами) на микросервисы, придерживаясь подхода database-per-service. Спроектировать и реализовать микросервисную архитектуру, обеспечив взаимодействие сервисов через API Gateway. В результате должно получиться полностью работающее API, разбитое на независимые сервисы.

Ход работы

1. Анализ монолитного приложения

Исходное монолитное приложение включало следующие функциональные блоки:

- Управление пользователями и аутентификация
- Управление рецептами и шагами приготовления
- Социальные функции (комментарии, лайки, сохранения, подписки)

База данных монолита содержала таблицы: user, recipe, step, comment, like, saved_recipe, subscription.

2. Проектирование микросервисной архитектуры

Было принято решение выделить три микросервиса:

Микросервис	Порт	Ответственность
User Service	3001	Регистрация, аутентификация, управление профилями пользователей
Recipe Service	3002	CRUD рецептов, шаги приготовления
Social Service	3003	Комментарии, лайки, сохранения, подписки

API Gateway (порт 3000) – единая точка входа для всех клиентских запросов.

3. Реализация микросервисов

3.1 User Service (порт 3001)

Модели:

- User – id, username, email, password, avatar_url, bio, created_at

Эндпоинты:

Метод	URL	Описание
POST	/api/auth/register	Регистрация пользователя
POST	/api/auth/login	Вход, выдача JWT
GET	/api/auth/me	Профиль (требуется токен)
PUT	/api/auth/me	Обновление профиля
GET	/internal/users/:id	Внутренний эндпоинт для других сервисов

Особенности:

- Пароли хешируются с помощью bcrypt
- JWT-токен выдаётся при регистрации и входе
- Внутренний эндпоинт используется Recipe Service и Social Service для проверки существования пользователя

3.2 Recipe Service (порт 3002)

Модели:

- Recipe – id, title, description, cooking_time, difficulty, category, image_url, video_url, author_id, created_at
- Step – id, recipe_id, step_number, instruction

Эндпоинты:

Метод	URL	Описание
GET	/api/recipes	Список рецептов (+ статистика из Social Service)
GET	/api/recipes/:id	Детали рецепта + шаги + статистика
POST	/api/recipes	Создание рецепта
PUT	/api/recipes/:id	Обновление рецепта (только автор)
DELETE	/api/recipes/:id	Удаление рецепта
GET	/api/recipes/:recipeId/steps	Шаги рецепта
POST	/api/recipes/:recipeId/steps	Добавить шаг
GET	/internal/recipes/:id	Внутренний эндпоинт

Межсервисное взаимодействие:

- При получении списка рецептов Recipe Service обращается к Social Service через внутренний эндпоинт /internal/recipes/:id/stats для получения количества лайков и комментариев.
- При недоступности Social Service возвращаются нулевые значения (graceful degradation).

3.3 Social Service (порт 3003)

Модели:

- Like – id, user_id, recipe_id, created_at
- Comment – комментарии (id, text, user_id, recipe_id, created_at)
- SavedRecipe – сохранённые рецепты (id, user_id, recipe_id, saved_at)
- Subscription – подписки (id, subscriber_id, author_id, created_at)

Эндпоинты:

Метод	URL	Описание
GET	/internal/recipes/:id/stats	Внутренний эндпоинт: получение количества лайков и комментариев для рецепта
GET	/comments/recipe/:recipeId	Получение всех комментариев к рецепту
POST	/comments/recipe/:recipeId	Добавление комментария к рецепту (требуется JWT)
DELETE	/comments/:commentId	Удаление комментария (только автор, требуется JWT)
GET	/likes/recipe/:recipeId	Получение количества лайков рецепта
POST	/likes/recipe/:recipeId	Постановка лайка рецепту (требуется JWT)
DELETE	/likes/recipe/:recipeId	Удаление лайка с рецепта (требуется JWT)
GET	/saved/me	Получение списка сохранённых рецептов текущего пользователя (требуется JWT)

POST	/saved/recipe/:recipeId	Сохранение рецепта в избранное (требуется JWT)
DELETE	/saved/recipe/:recipeId	Удаление рецепта из сохранённых (требуется JWT)
GET	/subscriptions/me	Получение списка подписок текущего пользователя (требуется JWT)
POST	/subscriptions/user/:userId	Подписка на пользователя (требуется JWT)
DELETE	/subscriptions/user/:userId	Отписка от пользователя (требуется JWT)

Межсервисное взаимодействие:

- При добавлении комментария Social Service проверяет существование пользователя через User Service (GET /internal/users/:id) и существование рецепта через Recipe Service (GET /internal/recipes/:id)
- Аналогичные проверки выполняются для лайков, сохранений и подписок
- При недоступности сервисов возвращается ошибка 404

Вывод

В ходе выполнения лабораторной работы было успешно выполнено разделение монолитного бэкенд-приложения на три независимых микросервиса:

1. User Service – отвечает за пользователей и аутентификацию
2. Recipe Service – отвечает за рецепты и шаги приготовления
3. Social Service – отвечает за социальные функции

Ключевые результаты:

- Каждый сервис имеет собственную базу данных (database-per-service)
- Взаимодействие сервисов осуществляется через API Gateway
- Реализованы внутренние эндпоинты для межсервисного взаимодействия
- Сохранена JWT-аутентификация (единый секрет для всех сервисов)
- При недоступности Social Service Recipe Service продолжает работу (graceful degradation)